INFORME T?CNICO DE CALIDAD Y PRUEBAS DE SOFTWARE

PROYECTO MERCATTO (E-COMMERCE SYSTEM) ? SERVICIO DE PEDIDOS

Integrante	Rol de Sustentaci?n	Alcance T?cnico Evaluado
Integrante 1	Contextualizaci?n Te?rica y Alcance	Definici?n de Pruebas Unitarias, Pir?mide de Cohn y Aislamiento con Mocks
Integrante 2	Dise?o Metodol?gico de Casos	Matriz de Pruebas, Casos L?mite y Criterios de Aceptaci?n
Integrante 3	Implementaci?n T?cnica en C?digo	Patr?n AAA, Inyecci?n de Mocks y Aserciones con JUnit 5
Integrante 4	An?lisis de M?tricas y CI/CD	Tiempos, Cobertura, Detecci?n de Bugs y Pipeline en GitHub Actions

1. Introducci?n y Arquitectura de Mercatto

Mercatto opera sobre una arquitectura desacoplada: backend transaccional en Spring Boot (Java 17) con base de datos en **PostgreSQL (Supabase)**, frontend en React, pasarelas de pago (Wompi y Stripe) y modelo predictivo en Python. El flujo de pedidos es el componente m?s cr?tico; inconsistencias en inventario o transacciones causan p?rdidas financieras directas.

2. Clasificaci?n de Pruebas y Pir?mide de Cohn

Siguiendo la Pir?mide de Mike Cohn, se establecieron tres niveles: (1) Base con Pruebas Unitarias aisladas en la capa de servicios; (2) Capa media con Pruebas de Integraci?n y API REST; (3) Cima con Pruebas E2E. Para esta defensa se implement? el blindaje unitario de **PedidoService.java**.

3. Integrante 1: Fundamentaci?n Te?rica, Alcance y Aislamiento

Componente Evaluado: PedidoService orquesta 12 dependencias (repositorios de pedido, variante, producto, cup?n, pago y servicio de email). **Principio de Aislamiento (F.I.R.S.T.):** Con Mockito se a?slan las dependencias externas para evitar escrituras en **PostgreSQL (Supabase)** y consumos de red a Stripe o Wompi.

Pregunta de Examen Individual: *?Por qu? usar Mocks en lugar de probar contra PostgreSQL en Supabase?*
Respuesta de Defensa: Una prueba unitaria debe ser determinista y correr en milisegundos. Probar contra PostgreSQL en la nube (Supabase) introducir?a latencia de red, dependencia de conexi?n a Internet y fallas de red no atribuibles al c?digo Java.

4. Integrante 2: Dise?o de Escenarios y Matriz de Pruebas

ID	Escenario	T?cnica	Entrada / Mock	Resultado Esperado
TC-01	Compra Exitosa (Happy Path)	Partici?n V?lida	Email v?lido, Stock=10, Cant=2	200 OK - PedidoResponse PENDIENTE
TC-02	Quiebre de Stock (L?mite)	Valor L?mite	Stock=2, Solicitado=5	Lanza BadRequestException
TC-03	Email Vac?o / Espacios	Entrada Inv?lida	emailInvitado = ' '	Lanza BadRequestException inmediata
TC-04	Cup?n Inexistente	Excepci?n Negocio	codigoCupon = 'NO_EXISTE'	Lanza ResourceNotFoundException

Pregunta de Examen Individual: *?Por qu? probar casos negativos si el frontend ya valida?*
Respuesta de Defensa: Principio de **Defensa en Profundidad**. Un atacante puede omitir el frontend con Postman o curl; el backend en Spring Boot debe validar sus invariantes de forma aut?noma.

5. Integrante 3: Implementaci?n T?cnica en C?digo (JUnit 5 + Mockito)

```
@ExtendWith(MockitoExtension.class)
public class PedidoServiceTest {
    @Mock private PedidoRepository pedidoRepo;
    @Mock private VarianteRepository varianteRepo;
    @Mock private EmailService emailService;
    @InjectMocks private PedidoService pedidoService;

    @Test
    @DisplayName('TC-03: Rechaza pedido si el email de invitado tiene solo espacios')
    void deberiaLanzarExcepcionCuandoEmailEsInvalido() {
        // 1. ARRANGE
        PedidoRequest req = new PedidoRequest();
        req.setEmailInvitado('   ');

        // 2. ACT & 3. ASSERT
        assertThrows(BadRequestException.class, () -> pedidoService.crearPedidoInvitado(req));
        verify(pedidoRepo, never()).save(any()); // Certifica cero escrituras corruptas
    }
}
```

6. Integrante 4: An?lisis de M?tricas, Hallazgos y CI/CD

M?tricas: Ejecuci?n en 138 ms, 100% ?xito, 92% cobertura en m?todo evaluado. **Bug Mitigado en Mercatto:** Se corrigi? la validaci?n de email con '.trim().isEmpty()' evitando ?rdenes hu?rfanas. **CI/CD:** Integrado en GitHub Actions con 'mvn clean test' como Quality Gate impidiendo despliegues defectuosos a Azure.

7. Conclusiones

1. Los dobles de prueba blindan el flujo de compras de Mercatto sin depender de PostgreSQL ni pasarelas externas. 2. Las pruebas unitarias automatizadas act?an como especificaci?n viva de las reglas de negocio. 3. La integraci?n con GitHub Actions garantiza que ning?n c?digo con regresiones llegue a producci?n.